VSEPR-SIM Version Report

State Vector Audit — Core Implementation Review
F/U Check Analysis — Update Loop Survey — Beta-6 Repair Manifest

`v4.0.4.05` ⟶ `v5.0.0-beta6`

VSEPR-SIM Development — Internal Technical Document

2026 — Generated pre-refactor

Contents

1 Document Purpose

This document is the mandatory pre-refactor audit for the `v5.0.0-beta6` migration. It describes:

1. The core-of-core implementation: what the engine actually does right now.
2. The full state vectors for both the atomistic and coarse-grained bead layers.
3. Force (**F**) and energy (**U**) check mechanisms as implemented.
4. The update loop structure and ensemble types.
5. The macro-solving methods available.
6. All detected defects and inconsistencies from the audit.
7. The immediate action table with priority classification.

This document is **not aspirational**. It describes what exists, not what is planned. Planned changes are listed only in Section 8.

2 Core-of-Core: What the Engine Actually Does

2.1 The Fundamental Architecture

VSEPR-SIM is structured as a layered state-transformation engine:

$$S_{n+1} = \mathcal{K}(S_n,\, \Delta t,\, \Lambda)$$

where $\mathcal{K}$ is the kernel operator, Δt is the timestep, and $\Lambda \in \{\text{C-Empirical}, \text{C-Reactive}, \text{C-Polarizable}, \text{CG-Red}$ is the model fidelity level.

The layers are:

Layer	Namespace	Role
Primitive math	`vsepr::`	Vec3, geom_ops, flat coord arrays
Atomistic state	`atomistic::`	State, StateC, integrators, models
Energy kernel	`vsepr::pot::`	Bond, angle, torsion, LJ, VSEPR domain
Optimizer	`vsepr::`	FIRE (geometry minimisation)
Coarse-grain	`coarse_grain::`	Bead, BeadSystem, mapping
V5 transport	`v5::`	BeadDynState, transport shell
Thermodynamics	`vsepr::thermo::`	Gibbs, ThermoDatabase

2.2 Two Parallel Engine Tracks

There are **two distinct coordinate conventions** operating simultaneously.

Track A: `src/sim + src/pot`
Coordinates as flat `std::vector<double>`, stride 3. Used by: FIRE optimizer, bond/angle/torsion/LJ/VSEPR energy kernels. This is the VSEPR geometry engine.

Track B: `atomistic/`
Coordinates as `std::vector<Vec3>`. Used by: velocity Verlet, Langevin, FIRE (atomistic variant), LJ+Coulomb MD model. This is the molecular dynamics engine.

Both tracks define their own `Vec3`. They do not interoperate. This is the primary structural defect of the current codebase.

3 State Vectors

3.1 Atomistic Runtime State: `atomistic::State`

The canonical per-step simulation state. Lives in `atomistic/core/state.hpp`.

$$S = (N,\ \mathbf{X},\ \mathbf{V},\ \mathbf{T},\ \mathbf{Q},\ \mathbf{M},\ \text{type},\ B,\ L,\ \mathbf{F},\ E,\ \text{box},\ \boldsymbol{\mu}^{\text{ind}},\ \boldsymbol{\alpha},\ \boldsymbol{\mu}^{0})$$

Symbol	Field	Type	Units
N	`N`	`uint32_t`	—
$\mathbf{X}$	`X[N]`	`Vec3[]`	Å
$\mathbf{V}$	`V[N]`	`Vec3[]`	Å/fs
$\mathbf{T}$	`T[N]`	`double[]`	K (proxy, optional)
$\mathbf{Q}$	`Q[N]`	`double[]`	e
$\mathbf{M}$	`M[N]`	`double[]`	amu
	`type[N]`	`uint32_t[]`	species ID
B	`B`	`Edge[]`	bond graph
L	`L`	`Event[]`	event log
$\mathbf{F}$	`F[N]`	`Vec3[]`	kcal/mol/Å (scratch)
E	`E`	`EnergyTerms`	kcal/mol
	`box`	`BoxPBC`	Å
$\boldsymbol{\mu}^{\text{ind}}$	`induced_dipoles[N]`	`Vec3[]`	e·Å
$\boldsymbol{\alpha}$	`polarizabilities[N]`	`double[]`	Å^3
$\boldsymbol{\mu}^{0}$	`permanent_dipoles[N]`	`Vec3[]`	e·Å

EnergyTerms ledger:

$$E_{\text{total}} = U_{\text{bond}} + U_{\text{angle}} + U_{\text{tors}} + U_{\text{vdW}} + U_{\text{Coul}} + U_{\text{ext}} + U_{\text{pol}}$$

All seven terms are individually tracked. The `total()` method sums them. U_{pol} is the SCF polarization energy, wired to the induced dipole solver.
BoxPBC: Orthogonal minimum-image convention. $\Delta\mathbf{r}_{ij} \leftarrow \Delta\mathbf{r}_{ij} - \mathbf{L}\cdot\text{round}(\Delta\mathbf{r}_{ij}/\mathbf{L})$, component-wise. Cached inverse $\mathbf{L}^{-1}$ avoids repeated division. Triclinic cells: not supported.

3.2 Physics-Informed Summary State: `atomistic::StateC`

Formal notation (Dev Day ∼42):

$$S_0 = \{m_0,\ E_0,\ \mathbf{x}_0,\ \mathbf{v}_0,\ C,\ \mathcal{E},\ K,\ \Sigma\}$$

$$S_f = \{m_f,\ E_f,\ \Phi,\ \mathcal{S},\ \Pi,\ \Omega,\ Q\}$$

$$S_f = T(S_0,\ t,\ \Lambda)$$

Symbol	Struct	Content
C	`Composition`	Atomistic / Bead / Hybrid / Macro
$\mathcal{E}$	`Environment`	T, P, **E**-field, **B**-field, radiation, shear, flow
K	`Constraints`	Frozen atoms, bond break/form, symmetry
Σ	`SourceTag`	Origin, engine version, SHA-512 hash, parent ID
Λ	`ModelLevel`	6-tier fidelity ladder
Φ	`StructuralState`	Geometry class, topology, symmetry score
$\mathcal{S}$	`StabilityMetric`	E/N, F_{RMS}, $F_{\max}$, local-minimum flag
Π	`PerformanceProperties`	D, η, κ, K_{bulk}, ρ, T_m
Ω	`EventLog`	Step-tagged events
Q	`QualityFlags`	Conservation booleans, drift values

3.3 Coarse-Grained Bead State: `coarse_grain::Bead`

Field	Type	Role
`position`	`Vec3`	Bead center (Å)
`velocity`	`Vec3`	Bead velocity (Å/fs)
`mass`	`double`	$\sum m_i^{\text{parent}}$ (amu)
`charge`	`double`	$\sum q_i^{\text{parent}}$ (e)
`type_id`	`uint32_t`	Index into BeadType table
Σ_i	`StructuralRole`	Inert / Ionic / Covalent / Metallic / Mixed
Λ_i	`StabilityClass`	Transient / Metastable / AmbientStable /
`parent_atom_indices`	`uint32_t[]`	Traceback into `atomistic::State`
`mapping_rule_id`	`uint32_t`	Producing mapping rule
`com_position`	`Vec3`	Center-of-mass reference
`cog_position`	`Vec3`	Center-of-geometry reference
`mapping_residual`	`double`	|COM − COG| (Å)
`surface`	`optional<SurfaceDescriptor>`	Anisotropic shape
`multi_channel`	`optional<MultiChannelDescriptor>`	Enriched descriptor
`unified`	`optional<UnifiedDescriptor>`	Adaptive-resolution descriptor
`orientation`	`BeadOrientation`	Inertia-frame primary axis
`has_orientation`	`bool`	Orientation validity flag

3.4 V5 Transport Bead State: `v5::BeadDynState`

Dynamic state for the V5 transport testbed. Equation of motion:

$$\frac{d\mathbf{v}}{dt} = A(\mathbf{u} - \mathbf{v}) + B\,\mathbf{G} + C\,\mathbf{H}$$

Field	Type	Content
`position`, `velocity`, `acceleration`	`Vec3`	Full kinematic state
`n_hat`	`Vec3`	Orientation axis (static, Phase A/B)
`env`	`EnvironmentState`	(η, ρ, C, P_2)
`mass`	`double`	Reduced mass (default 1.0)
`a_drag`, `a_body`, `a_grad`, `a_total`	`Vec3`	Decomposed acceleration (diagnostics)
`mod_report`	`ModulationReport`	Kernel modulation diagnostics

4 F and U Checks

4.1 Force Checks (F)

4.1.1 Analytical Gradient Verification

Available in `src/sim/optimizer.hpp` via `check_gradients = true`. Central finite difference:

$$\nabla_i E \approx \frac{E(\mathbf{x} + h\hat{e}_i) - E(\mathbf{x} - h\hat{e}_i)}{2h}$$

$h = 10^{-6}$ Å, tolerance 10^{-5} kcal/mol/Å. **Known bug:** the print path on max-error reporting recalls `evaluate_energy(coords)` instead of printing the stored numeric gradient, so the numeric column prints total energy. Diagnostics only — non-fatal, but misleading.

4.1.2 NaN / Inf Guard

FIRE optimizer checks every iteration:

```
if (has_invalid_values(result.coords) ||
    has_invalid_values(forces) ||
    !std::isfinite(result.energy))
    result.termination_reason = "NaN/Inf detected";
```

Terminates cleanly. Does not repair.

4.1.3 Force Convergence Criteria

Two simultaneous thresholds in `OptimizerSettings`:

$$F_{\text{RMS}} < 10^{-4} \frac{\text{kcal}}{\text{mol} \cdot \text{Å}}, \quad F_{\max} < 10^{-3} \frac{\text{kcal}}{\text{mol} \cdot \text{Å}}$$

Both must be satisfied simultaneously. These are tight relative to many production codes.

4.1.4 Displacement Clamping

Per-atom displacement clamped to `max_step` = 0.2 Å per step (vector magnitude, not component-wise). Prevents runaway steps on stiff geometries.

4.1.5 FIRE Convergence: `atomistic` Track

In `atomistic/integrators/fire.hpp`:

$$F_{\text{RMS}} < \epsilon_F = 10^{-6}, \quad \Delta U/N < \epsilon_U = 10^{-10}$$

These are significantly tighter than the `src/sim` FIRE defaults and operate on different coordinate storage (Vec3 arrays vs flat doubles).

4.2 Energy Checks (U)

4.2.1 EnergyBalance Ledger (`StateC`)

Enforces:

$$E_0 + E_{\text{in}} - E_{\text{out}} \approx E_f + E_{\text{loss}}, \quad m_0 + m_{\text{in}} - m_{\text{out}} \approx m_f$$

Violation sets `Q.energy_conserved = false` and increments `Q.warning_count`. Does **not** halt simulation — violation is logged, not fatal.

4.2.2 EnergyTerms Completeness

All seven terms are present in the ledger. `total()` sums all seven. The VSEPR-mode EnergyResult struct also carries the full breakdown: bond, angle, torsion, nonbonded, VSEPR domain, vdW, Coulomb.

4.2.3 Conservation in Velocity Verlet

NVE conserves $E_{\text{total}} = KE + PE$ by construction (symplectic integrator). Tracked in `VelocityVerletStats`: `E_drift = E_final - E_initial`. No per-step assertion — drift accumulates silently unless the caller inspects stats.

4.2.4 Bead Conservation Checking

`BeadSystem::check_conservation()` compares $\sum m_b$ vs $\sum m_i^{\text{atom}}$ and $\sum q_b$ vs $\sum q_i^{\text{atom}}$ with tolerance 10^{-10}.

5 Update Loop Structure

5.1 Geometry Optimisation (FIRE, `src` track)

Used for VSEPR structure generation. Not a dynamics integrator — minimises $E(\mathbf{x})$.

1. Compute E, $\mathbf{F} = -\nabla E$ at current $\mathbf{x}$.
2. Check convergence (F_{RMS}, $F_{\max}$, NaN/Inf, $dt_{\min}$).
3. Compute power $P = \mathbf{F} \cdot \mathbf{v}$.
4. FIRE velocity mixing: $\mathbf{v} \leftarrow (1-\alpha)\mathbf{v} + \alpha|\mathbf{v}|\hat{\mathbf{F}}$.
5. Adaptive dt, α: if $P > 0$, grow dt, decay α; if $P \leq 0$, reset $\mathbf{v} = 0$, shrink dt, reset α.
6. Velocity Verlet half-step: $\mathbf{v}(t + \frac{\Delta t}{2}) = \mathbf{v}(t) + \mathbf{F}(t)\frac{\Delta t}{2}$.
7. Position update: $\mathbf{x}(t + \Delta t) = \mathbf{x}(t) + \mathbf{v}(t + \frac{\Delta t}{2})\Delta t$. Clamp displacement.
8. Evaluate E, $\mathbf{F}$ at new $\mathbf{x}$.
9. Velocity Verlet second half: $\mathbf{v}(t + \Delta t) = \mathbf{v}(t + \frac{\Delta t}{2}) + \mathbf{F}(t + \Delta t)\frac{\Delta t}{2}$.
10. Return to step 2.

5.2 NVE Molecular Dynamics (Velocity Verlet, `atomistic` track)

Symplectic, time-reversible, conserves E_{total} (microcanonical ensemble).

1. Half-kick: $\mathbf{v}_i \leftarrow \mathbf{v}_i + \dfrac{\mathbf{F}_i}{m_i}\dfrac{\Delta t}{2}$.
2. Drift: $\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i \Delta t$.
3. Force evaluation: `model.eval(state, params)` ⇒ fills `F[N]`, `E`.
4. Half-kick: $\mathbf{v}_i \leftarrow \mathbf{v}_i + \dfrac{\mathbf{F}_i}{m_i}\dfrac{\Delta t}{2}$.
5. Diagnostics: $KE = \frac{1}{2}\sum m_i|\mathbf{v}_i|^2$, $T = \frac{2KE}{3Nk_B}$, log.

5.3 NVT Molecular Dynamics (Langevin, `atomistic` track)

Stochastic equation of motion (Euler-Maruyama discretisation):

$$m_i \frac{d\mathbf{v}_i}{dt} = \mathbf{F}_i - \gamma m_i \mathbf{v}_i + \sqrt{2\gamma m_i k_B T}\,\mathbf{R}(t)$$

1. Evaluate forces: `model.eval(state, params)`.
2. Update velocity: $\mathbf{v}_i \leftarrow \mathbf{v}_i + [\mathbf{F}_i/m_i - \gamma\mathbf{v}_i]\Delta t + \sqrt{2\gamma k_B T/m_i}\sqrt{\Delta t}\,\mathbf{R}$.
3. Update position: $\mathbf{x}_i \leftarrow \mathbf{x}_i + \mathbf{v}_i\Delta t$.
4. Accumulate temperature statistics.

Not symplectic — energy is not conserved; temperature is controlled.

5.4 V5 Transport Loop (`v5::BeadDynState`)

Mandatory update order (ERB §):

1. Integrate poses ($\mathbf{R}$, $\hat{\mathbf{n}}$) via velocity Verlet.
2. Compute fast observables: ρ, C, P_2.
3. Update slow state η.
4. Evaluate modulated kernels $\mathcal{K}_k(\eta)$.
5. Compute forces and torques.
6. Return to step 1.

Torque evolution and orientation dynamics are deferred (Phase A/B: rigid orientation).

5.5 StateC Transformation Loop (`state_c.hpp`)

Status: stub. Framework is complete; integrator is not connected. Current loop body:

```
// [PLACEHOLDER] In the real engine, force evaluation, integrator,
// thermostat, PBC, and constraint enforcement happen here.
```

The energy balance ledger, quality flags, event log, and provenance chain are wired and functional. The force and position updates are not.

6 Simulation Types and Macro-Solving Methods

6.1 Available Simulation Types

Type	Ensemble	Integrator	Status	Track
VSEPR geometry opt.	— (minimisation)	FIRE	Implemented	`src/sim`
NVE MD	Microcanonical	Velocity Verlet	Implemented	`atomistic`
NVT MD	Canonical	Langevin	Implemented	`atomistic`
Geometry opt. (atomistic)	—	FIRE	Implemented	`atomistic`
V5 transport	Non-equilibrium	Velocity Verlet	Partial	`v5`
StateC evolution	Configurable	Stub	Not connected	`atomistic`
CG bead dynamics	—	Not defined	Not implemented	`coarse_grain`

6.2 Macro-Solving Methods

6.2.1 Gibbs Free Energy

$$G(T) = H_f - T \cdot S/1000$$

(H_f in kcal/mol, S in cal/mol·K.) Seeded NIST reference database: 15 molecules, standard state 298.15 K / 1 atm. Temperature correction is constant-C_p approximation ($\int C_p\,dT$ not integrated).

6.2.2 Equilibrium Constant

$$K_{\text{eq}} = \exp\left(-\frac{\Delta G}{RT}\right)$$

6.2.3 SCF Polarization

Iterative induced dipole solver:

$$\boldsymbol{\mu}_i = \alpha_i \left(\mathbf{E}_i^0 + \sum_{j \neq i} \mathbf{T}_{ij} \boldsymbol{\mu}_j \right)$$

Wired into `State.Upol` and `EnergyTerms`.

6.2.4 V4 Formation Engine

Statistical mechanics of metallic formation. Polynomial fitting (11–15 layer), eigen counters. Continual formation from candidate central atoms (Au, Ag, Ni, Al, Fe, Cr, Ti, Co). Version: `V4.0.4.04` (current), `V4.0.4.05` (audit target).

6.2.5 Gas-Phase Thermodynamics (`gas2`, `gas3`)

Equation of state, kinetic theory, heat, nuclear corrections. Sweep engine, polynomial fitting, quality scoring, state records.

7 Detected Errors and Inconsistencies

The following defects were identified during the pre-beta6 audit.

7.1 E-1 — Two Vec3 Types (BLOCKER)

Location: `src/core/math_vec3.hpp` (`vsepr::Vec3`) vs `atomistic/core/state.hpp` (`atomistic::Vec3`).

Problem: Both types represent $\mathbb{R}^3$ with identical semantics but are not interconvertible. Every interface between the energy kernel (Track A) and the atomistic integrators (Track B) requires either a copy, a cast, or a parallel reimplementation. This is not a latent bug — it is an active barrier to every planned cross-track integration.

Impact:

- Coarse-grain beads import `atomistic::Vec3` for position/velocity.
- VSEPR energy kernels use `vsepr::Vec3` exclusively.
- No conversion path exists.
- Any function taking a bead position and feeding it to an energy kernel requires a full struct copy at the call site.

Classification: Blocker. Fix before refactor.

7.2 E-2 — `evolve()` Placeholder (BLOCKER)

Location: `atomistic/core/state_c.hpp`, `evolve()` function.

Problem: The canonical state-transformation operator $T(S_0, t, \Lambda)$ contains a `[PLACEHOLDER]` comment where the integrator coupling belongs. The loop runs for `n_steps` but does not move particles or evaluate forces. The energy balance always reports zero drift because the energy never changes.

What the stub does correctly:

- Validates S_0 (mass > 0, energy finite).

- Initialises the energy balance ledger.
- Chains provenance on each step.
- Returns a structurally valid `OutcomeState`.

What it does not do:

- Evaluate forces.
- Update positions or velocities.
- Advance the simulation by Δt.

Classification: Blocker. Core transformation is fiction without this.

7.3 E-3 — `sane()` Ignores `F[N]` Size

Location: `atomistic/core/state.hpp`, `sane()` function.
Problem:

```
inline bool sane(const State& s) {
    if (s.N == 0) return false;
    if (s.X.size() != s.N || s.V.size() != s.N ||
        s.Q.size() != s.N || s.M.size() != s.N ||
        s.type.size() != s.N) return false;
    return true;
}
```

`F[N]` is not checked. The force array can be empty or wrong size and `sane()` returns `true`. `model.eval()` writes into `F` — if the array is undersized, this is undefined behaviour.
Classification: Medium. Cheap fix, prevents silent UB.

7.4 E-4 — `T[N]` Has No Update Rule

Location: `atomistic/core/state.hpp`, field `T[N]`.
Problem: The per-particle temperature proxy `T[N]` is declared in `State` but nothing in the visible codebase populates or updates it. The correct system temperature is computed in `velocity_verlet.hpp` via the equipartition theorem:

$$T = \frac{2KE}{3Nk_B}$$

but is not written back into `T[N]`.
Classification: Medium. Mark as diagnostic-only until a thermostat or local-temperature operator is implemented.

7.5 E-5 — Bead Mass Not Bridged to Transport Layer

Location: `v5/bead_transport.hpp`, `BeadDynState::mass`.
Problem: `BeadDynState.mass = 1.0` (reduced units, default). Physical bead mass is:

$$M_b = \sum_{i \in P_b} m_i$$

stored correctly in `coarse_grain::Bead::mass` (amu). No conversion bridge exists between

the physical mass and the reduced mass used in the transport testbed. The equation of motion $\mathbf{a} = \mathbf{F}/m$ in the transport layer uses a mass that is not physically connected to the source atomistic data.

Classification: Medium. Reduced units are acceptable; the conversion must be explicit and stored.

7.6 E-6 — `StateC::from_state()` Loses Metadata

Location: `atomistic/core/state_c.hpp`, `StateC::from_state()`.

Problem: The factory correctly computes m_0, E_0, COM position, COM velocity, and sets `comp = Atomistic`. It does not transfer:

- `env` (Environment: temperature, pressure, fields, flow)
- `constraints` (Constraints: frozen atoms, bond rules, symmetry)
- `provenance.engine_version`, `.hash`

The compressed state silently loses physical context. Any downstream use of `StateC.env.temperature` after `from_state()` returns 0 K.

Classification: Design gap. Silent omission. Must be flagged at minimum.

7.7 E-7 — Bead Orientation Duplicated Across Layers

Location: `coarse_grain/core/bead.hpp` (`Bead::orientation`), `v5/bead_transport.hpp` (`BeadDynState::n_hat`).

Problem: Orientation is stored in two separate structs with no defined synchronisation. `Bead::orientation` is the inertia-frame static axis. `BeadDynState::n_hat` is the transport-layer orientation axis, initialised from the scene bead but thereafter independently tracked. After any dynamic step, the two diverge with no update protocol.

Classification: Medium. Two truths = future bug shrine. Define one owner.

7.8 E-8 — Gradient Verifier Print Bug

Location: `src/sim/optimizer.hpp`, `verify_gradients()`, line 363.

Problem: The numeric gradient diagnostic print re-calls `model.evaluate_energy(coords)` and prints the total energy, not the numeric gradient component at the max-error index. Only affects debug output when `check_gradients = true`.

Classification: Low. Diagnostics only.

7.9 E-9 — TBP Axial/Equatorial Classification is Frame-Dependent

Location: `src/pot/vsepr_geometry.hpp`, `TRIGONAL_BIPYRAMIDAL` case.

Problem: Classification of axial vs equatorial positions uses $|dz_i|/r_i > 0.8$ — a z-axis alignment test. This breaks for any trigonal bipyramidal molecule that is not pre-aligned to the z-axis. Since the FIRE optimizer does not enforce frame alignment, the equatorial/axial assignment is not invariant under rotation.

Classification: High correctness risk. Fix requires frame-independent classification (e.g., via inertia tensor decomposition).

7.10 E-10 — VSEPR Domain Gradient Jacobian Approximation

Location: `src/pot/energy_vsepr.hpp`, lines 134–141.

Problem: The gradient propagation from domain-repulsion through bond-pair directions approximates the full Jacobian $\partial\hat{\mathbf{d}}/\partial\mathbf{r} = (I - \hat{\mathbf{d}}\hat{\mathbf{d}}^\top)/|\mathbf{r}|$ by using only the $1/r$ magnitude scaling. The transverse projection tensor $(I - \hat{\mathbf{d}}\hat{\mathbf{d}}^\top)$ is not accumulated. This is accurate near equilibrium where bond directions are slowly varying, but degrades when bonds are significantly stretched.

Classification: Medium. Document as known approximation.

8 Immediate Action Table

ID	Issue	Priority	Action
E-1	Two Vec3 types	**P1 Blocker**	Unify or bridge before refactor
E-2	evolve() stub	**P1 Blocker**	Wire minimal empirical loop
E-3	sane() ignores F[]	**P2**	Add F.size() == N check
E-4	T[N] undefined update	**P2**	Mark diagnostic-only
E-5	Bead mass = 1.0	**P2**	Add conversion bridge metadata
E-6	StateC metadata silent	**P2**	Add completeness flags
E-7	Orientation duplicated	**P2**	Define one owner, freeze other
E-9	TBP frame-dependent	**P2**	Inertia-tensor classification
E-8	Gradient verifier print	**P3**	Fix numeric print path
E-10	VSEPR Jacobian approx	**P3**	Document; fix post-stability

9 Pre-Refactor Checklist

Complete this list before touching the architecture:

- ☐ Freeze version tag `v4.0.4.05` with audit notes.
- ☐ Choose canonical Vec3 (`vsepr::Vec3` recommended as base).
- ☐ Add `atomistic::Vec3 = vsepr::Vec3` or explicit bridge.
- ☐ Wire minimal `evolve()` loop: validate → clear F → eval F → VV step → update ledger → quality check → log events.
- ☐ Add `F.size() == N` check in `sane()`.
- ☐ Add `(T.empty() || T.size() == N)` check in `sane()`.
- ☐ Document `T[N]` as diagnostic-only (no update rule yet).
- ☐ Document `A[N]` as intentional derived quantity ($\mathbf{a} = \mathbf{F}/m$, not stored).
- ☐ Add bead mass conversion bridge: `BeadDynState::mass_amu` field + scaling.
- ☐ Define orientation ownership: `Bead` = static/identity, `BeadDynState` = dynamic.
- ☐ Add metadata completeness flags to `StateC::from_state()`.
- ☐ Fix gradient verifier print (E-8).
- ☐ Version tags: `v4.0.4.05` (kernel audit), `v5.0.0-beta6` (UFX migration start).

10 Version Labels

Track	Tag	Description
v4 patch	`v4.0.4.04`	Current (post last bump)
v4 patch	`v4.0.4.05`	Kernel audit and UFX migration preparation
v5 beta	`v5.0.0-beta6`	Current (post last bump)
v5 beta	`v5.0.0-beta7`	Post-repair: P1 blockers resolved, P2 bridges added

Decision rule: beta6 is the audit snapshot. beta7 opens only after E-1 and E-2 are resolved and the checklist is complete.